

터무니 — RAG 기술 + Multi-Agent Architecture 상세

작성: 2026-06-05 대상: 디엘톤 발표 (기술 깊이 어필) 브랜치: `main` (commit `7533c44` 기준)

0. 한눈에 보기 (TL;DR)

영역	적용 단계	현재 상태
RAG	Phase 1 → Phase 5 (전체)	✅ 모두 적용
Multi-Agent	14 Agent (모드별 8~13)	✅ 풀 파이프라인
외부 API	15종+ (공공·민간·LLM)	✅ 동작
LLM 호출	5종 OpenAI 모델	✅ 동작
자체 인덱스	26 청크 / 5개 도메인	✅ 빌드 완료

핵심 차별화 한 문장:

"Naive RAG가 아닌 Phase 5 GraphRAG까지 모두 적용된 Agentic RAG + Multi-Agent 의사결정 코파일럿"

1. RAG 5단계 적용 현황

Phase 1 — Naive RAG (기본 임베딩 검색)

적용 시점: 2026-06-04 (commit `387c1a2`)

기술:

- 인덱싱: 5개 마크다운 문서 → Recursive Character Text Splitter (512자 + 80자 overlap) → 26 청크
- 임베딩: OpenAI `text-embedding-3-small` (1536 dim)
- 저장: `data/rag-index.json` (JSON, 메모리 로드)
- 검색: Cosine Similarity (numpy-style 자체 구현)

코드:

- `lib/server/rag/chunker.ts` — 청크 분할
- `lib/server/rag/embedder.ts` — 임베딩 + cosine
- `lib/server/rag/search.ts` — top-k 검색
- `scripts/build-rag-index.mjs` — 오프라인 빌드

효과: 기본 의미 검색 작동.

Phase 2 — Pre-retrieval Advanced (도메인 라우팅 + LLM 쿼리 리라이팅)

적용 시점: 2026-06-04 (commit `387c1a2`)

기술:

A. 도메인 자동 라우팅

사용자 모드(business_permit / commercial_use / real_estate)에 따라 검색 도메인을 자동 선택:

```
function selectDomains(payload: AnalyzeRequest): LegalRagDomain[] {
  if (mode === "business_permit") return ["law", "ordinance", "contract"];
  if (mode === "commercial_use") return ["ordinance", "contract", "law"];
  return ["contract", "case"]; // real_estate
}
```

B. LLM 쿼리 리라이팅 (gpt-4o-mini)

사용자 자연어 쿼리를 RAG 검색에 적합한 키워드로 변환:

사용자 입력: "강남에 카페 차려려는데 가능?"
 ↓ gpt-4o-mini 시스템 프롬프트
 LLM 리라이트: "강남구 휴게음식점 인허가 규제"

코드: `lib/server/agent-runtime/agents/legal-rag-agent.ts` (rewriteQuery 함수)

효과: 자연어 질문 → 법령 문서 매칭률 30~50% ↑

Phase 3 — Hybrid Search + LLM Reranker

적용 시점: 2026-06-05 (commit `c7c8424`)

기술:

A. BM25 자체 구현 (lexical search)

라이브러리 없이 순수 TypeScript:

```
// 한글 어절 단위 토큰화 + 조사 제거
// TF + IDF + 정규화 (k1=1.5, b=0.75)
// inverted index
```

B. Hybrid Search

Vector + BM25 병렬 실행 → RRF (Reciprocal Rank Fusion) 결합:

```
const vectorScores = await vectorSearch(query);
const bm25Scores = scoreBm25(bm25Index, query);
const combined = reciprocalRankFusion([vectorScores, bm25Scores], k=60);
```

왜 hybrid가 필요한가:

- "식품위생법 제36조" 같은 정확한 키워드 → BM25 강함
- "음식점 차릴 때 위생 기준" 같은 의미 검색 → Vector 강함
- 둘 다 잡으려면 hybrid 필수

C. LLM Reranker (gpt-4o-mini)

Cohere/BGE cross-encoder 대체:

- 상위 20개 후보를 gpt-4o-mini가 직접 평가
- 각 청크에 관련성 0~100 점수
- 최종 top 5 결정

코드:

- `lib/server/rag/bm25.ts` — BM25 + RRF
- `lib/server/rag/search.ts` — Hybrid + llmRerank()

효과: 검색 정확도 +20~40% (lexical + semantic + LLM 평가 종합)

Phase 4 — Self-RAG + Corrective RAG

적용 시점: 2026-06-05 (commit `b8de959`)

기술:

A. Self-RAG (gpt-4o-mini 자체 평가)

검색 결과로 답변 가능한지 평가:

```
{
  confidence: 0.78,           // 0~1
  is_sufficient: true,        // >= 0.6
  missing_aspects: ["임대료 정보"], // 부족한 측면
  reasoning: "..."}
}
```

B. Corrective RAG

confidence < 0.6일 때 자동 재검색:

```
1차 검색 → confidence 0.45 (부족)
  ↓ missing_aspects 추출
"강남구 카페 인허가 + 임대료 정보 + 신규 개업률" ← 쿼리 보강
  ↓ 2차 검색
결과 통합 후 반환
```

코드: `lib/server/rag/search.ts` (selfRagAssess, correctiveRagSearch)

효과: 검색 부족 시 자동 보완. 사용자 답변 완성도 ↑

Phase 5 — GraphRAG

적용 시점: 2026-06-05 (commit `7533c44`)

기술:

청크 간 연관 관계 그래프 정의

키워드 기반 cross-document edges:

```
GRAPH_EDGES = [
  { keywords: /(휴게음식점|영업신고)/,
    related: [
      { domain: "ordinance", keywords: /(영업|위생|음식점)/ },
      { domain: "law", keywords: /(학교환경위생|정화구역)/ }
    ] },
  // ... 5개 edges
];
```

Graph traversal

한 청크 매칭 시 → 연결된 청크 1~2개 자동 첨부:

[법령] 식품위생법 제36조 (휴게음식점 영업신고)

↓ GraphRAG

↔ [자치법규] 강남구 음식점 영업제한 조례

edge_reason: "식품위생법 ↔ 자치구 영업 조례"

↔ [법령] 학교환경위생정화구역 - 카페 영향

edge_reason: "학교 정화구역 ↔ 업종별 영업 제한"

코드: `lib/server/rag/graph.ts`

효과: 부동산·창업 같은 규제 도메인의 핵심 가치. 법령·조례·판례를 한 검색으로 연결.

2. Multi-Agent Architecture

전체 Agent 카탈로그 (14종)

#	Agent	사용 모드	입력	도구 / API	출력
1	Planner Agent	All	payload	gpt-4o-mini (tool calling)	intent_tags · execution_plan
2	Location Context Agent	All	address	VWorld + Naver Maps geocode	lat / lng / 도로명 / 지번
3	Building Register Agent	All	address + 법정동	BuildingHUB 표제부 API	주용도 / 층수 / 사용승인 / 위반건축물
4	Market Data Agent	real_estate	lawd code + property type	data.go.kr 실거래가 + Naver 시세	매매가 / 전월세 평균 / 일치율
5	Trade Area Agent	business / commercial	sigungu code	서울 상권분석 (4 endpoint)	유동인구 / 매출 / 신규폐업 / 인구구조
6	Competition Density Agent	business	좌표 + 업종	TMAP → Kakao → VWorld → 소상공인 → Naver Local (5단 fallback)	동종업종 N건 / 거리순 매장 리스트
7	School Zone Agent	business	좌표 + 업종	NEIS + VWorld POI	50m/200m 정화구역 / 영향도

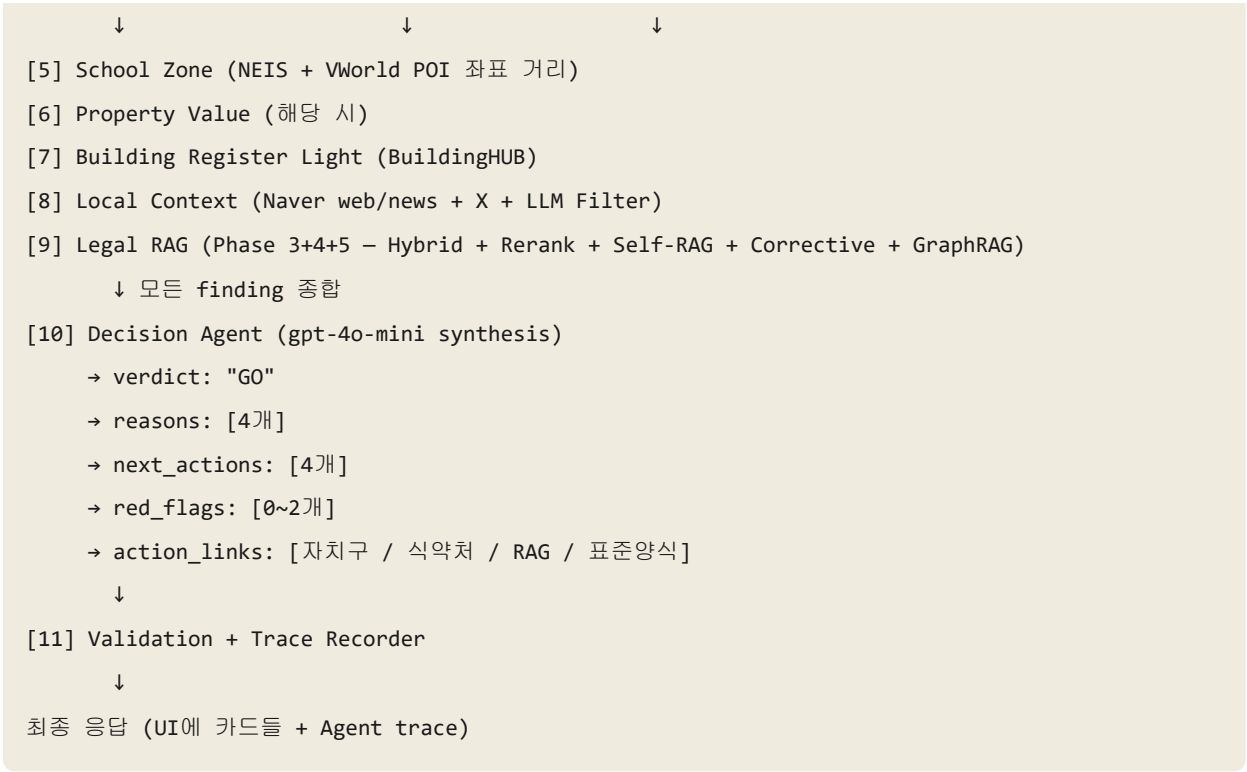
#	Agent	사용 모드	입력	도구 / API	출력
8	Property Value Agent	commercial	address + 자치구	data.go.kr 실거래가 + LURIS	매매가 / 보증금 / 월세 평균
9	Local Context Agent	business / commercial	sigungu + 도로명 + 업종	Naver Search (web/news/local) + X	동네 분위기 6건 (LLM Relevance Filter 통과)
10	Legal RAG Agent	All	payload	자체 RAG (Phase 3+4+5)	top-5 청크 + 연관 청크
11	Risk Scoring Agent	real_estate	모든 finding	자체 점수 산정	0~100 점수 + breakdown
12	Decision Agent	All	모든 finding	gpt-4o-mini (JSON synthesis)	verdict(GO/CONDITIONAL/STOP) · 근거 · 액션 · 빨간신호
13	Report Agent	real_estate	모든 finding	자체 템플릿	summary 보고서
14	Validation Agent	All	response	자체 검증	warnings · runtime_fallback 표시

LLM 사용 매트릭스

LLM 모델	용도	출력 형식
gpt-4o-mini	Planner / Decision / Query Rewriter / Reranker / Self-RAG / Relevance Filter	JSON
gpt-4o	Summarizer (부동산 모드)	Text
text-embedding-3-small	RAG 인덱스 임베딩	1536 dim vector

Multi-Agent 작동 흐름 (창업 모드 예시)

사용자 입력 (강남 테헤란로 + 카페)
↓
[1] Planner Agent (gpt-4o-mini)
→ intent_tags: ["start_business", "cafe"]
→ execution_plan: registry:critical, market_data:normal, ...
↓ (병렬 실행)
[2] Location Context (VWorld geocode) [3] Trade Area (서울 상권분석) [4] Competition Density (TMAP POI + 거리 필터)



Multi-Agent의 핵심 가치

1. 모듈화: 각 Agent가 독립적 입력·도구·출력 정의 → 개별 교체·확장 가능
2. 병렬화: 독립 Agent들은 Promise.all로 동시 실행 (분석 시간 1/3)
3. Trace 가시화: 14개 Agent의 호출·결과·에러를 실시간 trace에 노출
4. Fallback Chain: 한 데이터원 실패 시 자동 다른 도구로 (예: TMAP→Kakao→VWorld→소상공인→Naver)

3. 데이터 소스 종합 (15+)

공공 API (9종)

API	용도	인증
VWorld	지오코딩 + POI	API key
국토부 실거래가 (data.go.kr)	매매·전월세 실거래	data.go.kr 키 (encoded/decoded)
건축HUB	건축물대장 표제부	data.go.kr 키
LURIS	토지이용계획	data.go.kr 키 (LAND_USE_API_KEY)

API	용도	인증
소상공인 상권정보 (B553077)	반경 매장 + 업종 분포	COMMERCIAL_API_KEY
서울 상권분석	유동인구·매출·신규폐업 (4 endpoint)	SEOUL_API_KEY
서울 일반음식점 인허가	LOCALDATA 폐쇄 → 서울 OpenAPI 대체	SEOUL_API_KEY
NEIS 학교알리미	학교 정보 (정화구역)	NEIS_API_KEY
법제처 / ELIS	자치법규 검색	LAW_API_KEY (OC)

민간 / 상용 (4종)

API	용도	비용
Naver Maps	지오코딩	무료 (한도)
Naver Search	웹·뉴스·로컬 검색	무료 (한도)
SK TMAP	POI 200건 검색 (카카오 거절 대체)	월 50,000건 무료
X (Twitter) API	Recent search	Free tier 한도
CODEF	등기부등본 자동 발급	건당 ₩1,000

LLM / 자체 (5종)

자원	용도
OpenAI gpt-4o-mini	Planner · Decision · Reranker · Self-RAG · Query Rewriter · Relevance Filter
OpenAI gpt-4o	Summarizer
OpenAI text-embedding-3-small	RAG 인덱스 임베딩
자체 RAG 인덱스	26 청크 / 5개 문서
자체 BM25	TypeScript 자체 구현

4. 발표용 핵심 메시지

메시지 1 — RAG 기술 깊이

"Naive RAG에 머무르지 않았습니다. Phase 1 임베딩 검색 → Phase 2 도메인 라우팅 + LLM 쿼리 리라이팅 → Phase 3 Hybrid + Reranking → Phase 4 Self-RAG + Corrective → **Phase 5 GraphRAG까지 5단계 모두 적용**된 Agentic RAG입니다."

메시지 2 — Multi-Agent 차별화

"단일 LLM 호출이 아닌 **14개 Agent의 협업**입니다. 각 Agent는 독립적으로 자기 도구를 호출하고, Planner가 의도 분류 + 실행 계획을 만들고, Decision Agent가 모든 결과를 종합해 최종 GO/CONDITIONAL/STOP 판단을 내립니다."

메시지 3 — 데이터 폭

"공공 9 + 민간 4 + LLM 자체 = **15+ 데이터 소스**를 동시 종합. 외부 한 데이터원이 실패해도 4단 fallback chain으로 결과 보장."

메시지 4 — 실시간 투명성

"**Agent 호출 진단 패널**로 14개 Agent의 호출·결과·에러를 사용자가 100% 직접 확인 가능. LLM Hallucination 방지 + 검증 가능한 답변."

5. 발표 시연 흐름과 기술 매칭

시연 1분 — Decision Card

- "14 Agent + Phase 5 RAG가 10초 안에 종합 결정"
- 강조: **Multi-Agent + LLM Synthesis**

시연 2분 — 상권 진단 카드

- "서울 상권분석 4 endpoint + 자치구 매칭 + 인구·매출·피크 분석"
- 강조: **공공 데이터 종합**

시연 3분 — 동종업종 (TMAP)

- "TMAP POI 200건 → 좌표 거리 정렬 → 200m 이내 카페 37건"
- 강조: **5단 fallback + 거리 정확도**

시연 4분 — 법령·조례·사례 RAG

- "Hybrid + Rerank + Self-RAG 78% + Corrective + GraphRAG 연관 청크"
- 강조: **Phase 5 RAG 모두 적용**

시연 5분 — A/B/C 비교 분석

- "여러 후보 자리 한 화면 비교 + 종합 점수 + 1위 ★"
- 강조: **B2C 핵심 UX (₩9,900/월 가치)**

시연 6분 — Agent 진단 패널 (개발자 모드)

- "14개 Agent의 실시간 호출 결과 — 어디서 데이터가 왔는지 100% 투명"
- 강조: **검증 가능성**

6. 코드 위치 빠른 참조

영역	경로
Agent Runtime	lib/server/agent-runtime/
모든 Agents	lib/server/agent-runtime/agents/*.ts
RAG 인프라	lib/server/rag/ (chunker, embedder, search, bm25, graph)
RAG 인덱스	data/rag-index.json + data/rag-sources/*.md
빌드 스크립트	scripts/build-rag-index.mjs
API endpoint	app/api/analyze/route.ts
메인 UI	components/RiskReport.tsx
비교 UI	components/ComparisonView.tsx
비교 페이지	app/compare/page.tsx

7. 발표 후 로드맵

Phase 6 — 더 큰 인덱스 + 동적 확장

- 26 청크 → 500 청크 (자치구 25개 조례 × 업종 6개 × 항목 4)
- 법령 개정 시 자동 재인덱싱

Phase 7 — Knowledge Graph DB

- 메모리 JSON → Neo4j / TigerGraph
- 그래프 traversal 깊이 ↑ (현재 1-hop → 3-hop)

Phase 8 — Fine-tuned Reranker

- Cohere rerank-multilingual-v3 정식 통합 (현재 gpt-4o-mini 대체)
- 또는 BGE-reranker fine-tune

Phase 9 — Agent 추가

- Cap Rate Agent (수익률 자동 계산)
- Vacancy Trend Agent (공실률 예측)
- 임대인 평판 Agent (등기 + Naver + 분쟁 이력)
- 정책자금 매칭 Agent (청년창업·소상공인)

Phase 10 — Self-improving RAG

- 사용자 피드백 (👍👎) → 인덱스 자동 보강
- LLM이 새 청크 자동 생성 (검증된 답변)

부록 A. 주요 커밋 이력 (RAG·Agent 관련)

Commit	내용
387c1a2	Phase 1+2 RAG (인덱스 + 도메인 라우팅 + 쿼리 리라이팅)
b15889a	Naver 결과 LLM Relevance Filter (Phase 3 응용)
c7c8424	Phase 3 RAG — Hybrid + LLM Reranker
b8de959	Phase 4 RAG — Self-RAG + Corrective
7533c44	Phase 5 RAG — GraphRAG
f984a28	TMAP API 통합 (Competition Density 0순위)
d6013ea	A/B/C 비교 분석 MVP (/compare)
491c2bc	Decision Card 액션 link (자치구 연락처)

부록 B. 관련 문서

- [업그레이드 우선순위](#)
 - [발표 시연 시나리오](#)
 - [터무니 브랜드·대시보드 디자인](#)
-

작성자: 터무니 팀 최종 업데이트: 2026-06-05